REST API Design

Contents

REST API Design **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 3

REST API Design

Scope

Resource naming, HTTP semantics, payload shape and error format for the public API. Rationale: `brain/decisions/ADR-0005-api-style-rest-json.md`.

Official requirements

- Subject V.4: the API must support REST principles, use JSON as the exchange format, and carry reference documentation of methods, inputs and outputs. Both choices must be justifiable.
- Subject V.5: the backend address is configurable in the client, so the API must not assume a fixed origin.
- HTTP semantics per RFC 9110. Error bodies per RFC 9457 Problem Details.

Project policy

URLs

- Base path `/api/v1`. Version in the path; bump only for breaking changes.
- Plural, lower-case, kebab-case resource nouns: `/api/v1/playlists/{playlistId}/tracks`.
- No verbs in paths. An action that is not CRUD becomes a sub-resource: `POST /api/v1/events/{eventId}/tracks/{tr`
- Nest at most two levels. Deeper relationships get a top-level resource with a filter.

Methods and status codes

Method	Semantics	Success
`GET`	safe, idempotent, never mutates	`200`, `204`
`POST`	create or non-idempotent action	`201` + `Location`, or `200`
`PUT`	full idempotent replace	`200`, `204`
`PATCH`	partial update	`200`, `204`
`DELETE`	idempotent removal	`204`

Failure codes: `400` malformed, `401` unauthenticated, `403` authenticated but not permitted, `404` absent **or hidden from this caller**, `409` concurrency conflict, `422` semantically invalid, `429` rate limited.

Privacy rule: a private resource the caller may not see returns `404`, never `403`. `403` would confirm the resource exists.

Payloads

- JSON only, `camelCase` field names, UTF-8.
- Timestamps are UTC ISO-8601 with an offset: `2026-09-23T18:00:00Z`.
- Ids are strings in JSON even when numeric in the database.
- Collections return an envelope: `{ "items": [...], "page": 1, "pageSize": 20, "total": 137 }`.
- Every list endpoint is paginated with a hard maximum `pageSize`. There is no "all" option.
- Never return a field the caller is not allowed to see. Filter in the query, not in the client.

Errors

One shape everywhere, `application/problem+json`:

```
{
  "type": "https://musicroom.dev/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "detail": "trackId must be a valid YouTube video id",
  "instance": "/api/v1/events/12/tracks",
  "errors": { "trackId": ["invalid format"] },
  "traceId": "00-4bf92f...-01"
}
```

Error text is safe for a client to display and never leaks stack traces, SQL, or internal paths.

Client context headers

Mutating requests MUST carry `X-Client-Platform`, `X-Client-Device`, `X-Client-App-Version`. These feed the mandatory audit log. Missing headers on a mutating request are `400`.

Cross-origin and transport

- HTTPS only. HTTP redirects, and HSTS is set.
- CORS allow-list is configuration-driven because the client's backend URL is configurable.
- No API keys or tokens in query strings, except the documented SignalR handshake parameter.

Checklist

- ☐ Nouns only in paths, versioned base path
- ☐ Correct status code per outcome, `404` for hidden resources
- ☐ Every list paginated with a capped page size
- ☐ All errors are Problem Details with a `traceId`
- ☐ Client context headers required and validated on mutations

References

- RFC 9110 HTTP Semantics — https://www.rfc-editor.org/rfc/rfc9110
- RFC 9457 Problem Details — https://www.rfc-editor.org/rfc/rfc9457
- OWASP API Security Top 10 — https://owasp.org/API-Security/

Related

../openapi-documentation/STANDARD.md · ../../40-security/authentication-authorization/STANDARD.md · ../../10-backend/signalr-realtime/STANDARD.md